Relaxed Concurrent Counting Bloom Filters

Dhruv Mishra 2020296
Kshitij Bajaj 2020565

BTP report submitted in partial fulfillment of the requirements for the Degree of B.Tech. in Computer Science & Applied Mathematics and the Degree of B.Tech. in Computer Science & Artificial Intelligence

On April 29, 2024

BTP Track: Research Track

BTP Advisor
Dr. Bapi Chatterjee

Indraprastha Institute of Information Technology
New Delhi

Student's Declaration

We hereby declare that the work presented in the report entitled "Relaxed Concurrent Counting Bloom Filters" submitted by Dhruv Mishra & Kshitij Bajaj for the partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science & Artificial Intelligence and Computer Science & Applied Mathematics at Indraprastha Institute of Information Technology, Delhi, is an authentic record of our work carried out under guidance of Dr. Bapi Chatterjee. Due acknowledgments have been given in the report for all material used. This work has not been submitted anywhere else for the reward of any other degree.

…….
Dhruv Mishra

……...
Kshitij Bajaj

Place & Date: .....

Certificate

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

…… **Place & Date: …….**
Dr. Bapi Chatterjee

Abstract

This report presents an in-depth study and implementation of a novel data structure, the Relaxed Concurrent Counting Bloom Filter (RCCBF). The RCCBF is an extension of traditional Bloom Filters, specifically engineered for high-throughput and concurrent environments. By integrating relaxed counters, the RCCBF efficiently manages concurrent insertions and deletions without the need for locking mechanisms, thereby significantly enhancing performance. This study explores various aspects of RCCBFs, including their accuracy measures and performance in multithreaded contexts.

Keywords:

Bloom Filters, Counting Bloom Filters, Relaxed Counters, Concurrent Data Structures, Multithreading, Probabilistic Data Structures, Data Stream Processing

Acknowledgments

We would like to express our gratitude to Dr. Bapi Chatterjee for providing us with an opportunity to work on this project. We would also like to thank Utkarsh Pal for his constant support and contribution throughout the project. This project would not have been possible without their support and guidance.

Work Distribution

We explored different research papers and sources during literature analysis and later discussed them among ourselves. Bloom Filters & d-left Counting Bloom Filters were explored by ***Kshitij,*** while Counting Bloom filters and stable Bloom Filters were explored by ***Dhruv***. Scalable Bloom Filters and Relaxed Counters were explored by us together. Initial code writing and experimentation were divided and conducted to replicate and verify existing research results. We experimented with lock and lock-free counters, relaxed counters, counting bloom filters, and the effects of multithreading on these. Code writing, experimentation, and analysis for our proposed model, the Relaxed Concurrent counting bloom filter, were performed by us together.

Contents

1. **Introduction**
2. **Bloom Filters**
    a. Structure
    b. Probability of False Positives
    c. Optimal Value of Number of Hash Functions
3. **Counting Bloom Filter**
4. **Multithreaded Counting Bloom Filters**
5. **Approximate Counters and Relaxed Counters**
    a. Introduction
    b. Implementation
    c. Algorithm Description
    d. Advantages
    e. Relation to Load Balancing
6. **Relaxed Concurrent Counting Bloom Filters**
    a. Introduction
    b. Accuracy Measures
    c. Accuracy Testing
    d. Throughput and Scalability Testing
7. **Future Work**
8. **Conclusion**

Introduction

Bloom filters are space-efficient probabilistic data structures to test the membership of elements in a set. Membership queries are essential in caching systems, network routing, distributed systems, databases, etc., and Bloom filters provide a fast and space-efficient solution with a small probability of false positives. They do not store the elements themselves. Unlike other data structures used for maintaining sets (such as binary trees or hash tables), the space requirements for bloom filters are independent of the size of the elements. The presence of an element is defined by a predetermined number of bits. Hence, bloom filters perform better than the alternatives with increasing element size. Standard Bloom filters do not provide any deletion support or maintain the count of the number of times an element is added and can get saturated with enough insertions after a point. There are variations of bloom filters that target some of its limitations and improve the performance of the data structure. We will discuss the existing variations and our proposed variation, Relaxed Concurrent Bloom Filters, in this paper.

Bloom Filters

A Bloom filter is a space-efficient probabilistic data structure used to test whether a given element is a member of a set. It was introduced by Burton Howard Bloom in 1970. A Bloom filter is a bit array of m bits, initially all set to 0. There are k different hash functions. To insert an element into the bloom filter, first pass the element through each of the k hash functions. The obtained outputs would be k array positions. Set the bit at these positions to 1. To test whether an element is in the set, pass the element through each of the k hash functions and check the k array positions obtained as output. If any of the bits is 0, the element is definitely not in the set. If all the k bits are 1, either the element is a part of the set or all the k bits were set to 1 while inserting other elements, giving us a false positive. Removing an element from the bloom filter is not possible because one of the k positions might be common with some other element in the set.

Probability of False Positives

Assumption: Each hash function selects every array position with equal probability.

If m is the number of bits in the array (size of the bloom filter), then the probability that a certain bit is not set to 1 by a certain hash function while inserting an element is:

$$1 - \frac{1}{m}$$

If there are k independent hash functions, then the probability that the bit is not set to 1 during the insertion of an element is:

$$(1 - \frac{1}{m})^k$$

For large m,

$$(1 - \frac{1}{m})^k = ((1 - \frac{1}{m})^m)^{k/m} \approx e^{-k/m}$$

If we have inserted n elements, the probability that a certain bit is 0 is

$$(1 - \frac{1}{m})^{kn} \approx e^{-kn/m}$$

The probability that this bit is 1 is

$$1 - (1 - \frac{1}{m})^{kn} \approx 1 - e^{-kn/m}$$

We get a false positive when all k positions for an element not in the set are set to 1. The probability of all of them being 1 using the above result is:

$$\varepsilon = (1 - (1 - \frac{1}{m})^{kn})^k \approx (1 - e^{-kn/m})^k$$

Optimal Value of k

For a given m and n, the optimal value for k(that minimizes the false positive probability) is

$$k = \frac{m}{n} ln\, 2$$

Given n and the desired false positive rate ε, we can obtain the number of bits required, m, assuming the optimal value of k.

$$\varepsilon = (1 - e^{-(\frac{m}{n} ln\, 2)\frac{n}{m}})^{\frac{m}{n} ln\, 2}$$

On simplifying,

$$log_2 \varepsilon = -\frac{m}{n} ln\, 2$$

Rearranging,

$$m = -\frac{n\, log_2 \varepsilon}{ln\ 2}$$

Optimal bits per element are

$$\frac{m}{n} = -\frac{log_2 \varepsilon}{ln\ 2} \approx -1.44\, log_2 \varepsilon$$

And corresponding optimal k

$$k = -\frac{ln\ \varepsilon}{ln\ 2} = -log_2 \varepsilon$$

So, to maintain a false positive probability of 1%, k = 6.64 and m/n = 9.6

That is, to maintain a false positive probability of 1%, we would require at least 9.6 bits/counters per element inserted.

Counting Bloom Filters

A Counting Bloom Filter (CBF) is an extension of the traditional Bloom Filter (BF) that introduces the concept of counters associated with each element. The purpose of this extension is to address limitations inherent in standard Bloom Filters, particularly regarding dynamic updates and element deletion.

Let m denote the number of bits in the Bloom Filter, n denote the number of elements inserted into the filter, and k denote the number of hash functions used for element insertion and querying.

Initialization Phase:
Each counter C_i associated with the i-th element is initialized to zero.

Element Insertion:
When an element x is inserted, the counters associated with x are incremented by one:
$C_{h1(x)}$, $C_{h2(x)}$, … , $C_{hk(x)}$ where $h_i(x)$ represents the i-th hash function.

Element Deletion:
Deleting an element x involves decrementing its associated counters:
$C_{h1(x)}$, $C_{h2(x)}$, … , $C_{hk(x)}$. This feature enables the removal of elements from the filter.

Membership Testing:
To test if an element x is in the set, all associated counters are examined. If any counter is less than the threshold θ, x is considered absent; otherwise, it is deemed a probable member based on the counter values.

False Positive rate:
Probability that one of the counter counts L is:

$$b(L, kn, \frac{1}{m}) = (C_l^{kn})(\frac{1}{m})^l(1 - \frac{1}{m})^{kn-l}$$

$$P_{fp}(\theta, k, n, m) = (1 - \sum_{L<\theta} b(L, kn, \frac{1}{m}))^k$$

Applications:
Counting Bloom Filters are utilized in scenarios requiring dynamic updates, frequency estimation, and efficient membership testing, such as network traffic monitoring, cache management, and data stream processing.

Multithreaded Counting Bloom Filters:

Multithreading is employed in Counting Bloom Filters (CBFs) to enhance throughput via concurrent accesses and updates, while maintaining accuracy in terms of false positive and false negative rates. Effective concurrency management is crucial to ensure the consistency of these rates with those of a single-threaded implementation, assuming proper synchronization.

Two primary approaches for parallelizing CBF operations are identified:

1.) **Parallel Computation for a Single Element:**

- Each element's hashes are computed concurrently, focusing on a single element at a time for insertion and contains queries.
- Advantages: Efficient use of available threads, no query reordering needed as operations are processed in order.
- Limitations: Maximum thread usage is capped by the number of hash functions per element.

2.) **Thread-per-Element Handling:**

- Multiple elements are processed simultaneously by assigning each element to a thread for insertion and contains queries.
- Advantages: Allows indefinite extension of threads, potentially enhancing concurrency.
- Limitations: Requires query reordering due to lack of thread synchronization.

A hybrid approach can combine these methods, employing multiple threads for hash function computation and seamlessly extending concurrency to handle multiple elements if thread limits are reached.

In our specific multithreaded CBF implementation:

- We parallelize the number of elements being inserted or checked, dedicating one thread to each element.
- Thread pinning is utilized to prevent thread migration and associated performance penalties.
- Our results demonstrate linear scalability of throughput with the increasing number of threads, aligning with our expectations.

Approximate Counters and Relaxed Counters:

Introduction:
Relaxed counters represent a specialized form of approximate counters tailored to enhance performance within a multithreaded context. They introduce a degree of tolerance for minor inaccuracies in counter values to yield substantial performance enhancements, chiefly by diminishing the necessity for locking mechanisms. This approach stands out as especially advantageous compared to conventional multithreading models, where stringent accuracy requirements and frequent locking can impede performance.

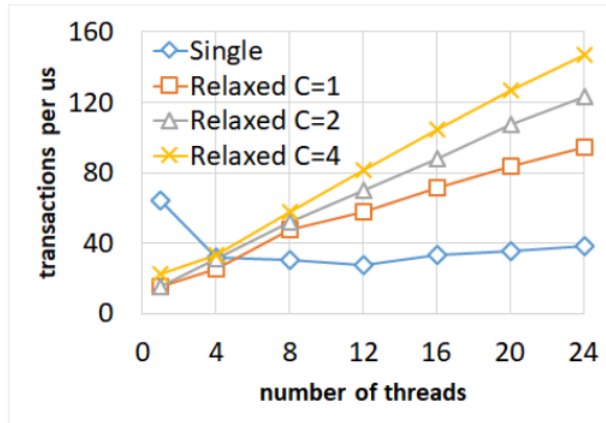
Our Implementation:
Delving into our existing relaxed counter implementation, let's envision a scenario with n threads that jointly utilize a set of m distinct atomic counters. The objective here is to distribute contention evenly across these m individual instances. To increment the global counter, a thread randomly selects two atomic counters, denoted as i and j, reads their values, and then atomically increments the counter with the lower value among the two.

For reading the global counter, the thread retrieves the value of a randomly chosen counter, i, and multiplies it by m. This process bears similarities to the classical two-choice load balancing procedure, where a sequence of balls is placed into m initially empty bins, and, at each step, a new ball is assigned to the less burdened of two randomly selected bins. In our scenario, the individual atomic counters serve as the bins, and each increment corresponds to the addition of a new "ball."

A sequence of in-depth technical findings has demonstrated that the difference between the most heavily loaded bin and the average is O(log logm) on average, and this difference remains stable as the process progresses over numerous iterations. Consequently, we can anticipate the relaxed concurrent counter described above to exhibit relatively low and consistent discrepancies among successive operations while also scaling effectively due to the even distribution of contention across the m counters.

Algorithm Description:
The algorithm implements an approximate counter by distributing updates across m distinct counters, each of which supports atomic read and increment operations. To retrieve the counter value, a thread simply selects one of the m counters uniformly at random and returns its value multiplied by m. When incrementing the counter value, the thread randomly selects two counter indices, i and j, and sequentially reads their current values. It then proceeds to update (increment) the value of the counter that appears to have the lower value based on the two reads. In cases of ties or identical choices, the tie-breaking process is arbitrary.

Advantages Over Traditional Models:
The conventional model of concurrent operations often relies on locking mechanisms to maintain consistency, which can become a bottleneck in a multithreaded environment. In contrast, approximate counters, such as relaxed counters, facilitate concurrent updates without the need for locks, resulting in significant reductions in the time and resources required for synchronization. This approach proves particularly beneficial when exact count accuracy takes a backseat to the performance gains achieved through relaxed consistency.



(a) Scalability of the concurrent counter for different values of the ratio C between counters and # threads.

Relation to Load Balancing:
The above description outlines a method that can be seen as a parallel to the well-known two-choice balanced allocation process. In this process, each counter is likened to a bin, and each increment represents a new ball being placed into the bin that is less full out of two randomly selected bins. This analogy holds true in a sequential version, where the counter's read or increment actions are done atomically.
However, in a concurrent setting, this process diverges significantly from its sequential counterpart. The key difference lies in the reliability of the values read during the process. In a concurrent environment, the values read may not align with those that would be observed in a sequential execution. It's possible that there's no specific moment when the counters show the values observed by a thread. Additionally, these values might change in the time between when they are read and when an update occurs.
In more technical terms, the sequential version of the two-choice process has an important characteristic: with each increment step, it tends to favor the counter with the lower value. This principle, however, may not apply to the concurrent approximate counter. In situations where a large number of updates happen between reading and updating, the information that was read becomes outdated. Consequently, the decision made by the thread on which counter to increment may essentially be as random as a purely chance decision.

Relaxed Concurrent Counting Bloom Filter:

Introduction:
Counting Bloom filters presents an extension of the Bloom filter concept, accommodating dynamic set changes through insertions and deletions. Instead of bits, they employ an array of counters to monitor the quantity of elements hashed to specific locations. When harnessed in multithreaded environments to enhance throughput, they introduce their own set of challenges, primarily stemming from conflicts among threads.

The integration of relaxed counters into a multithreaded counting Bloom filter can significantly boost its performance in concurrent scenarios, effectively surmounting these challenges. By leveraging relaxed counters, the Counting Bloom Filter (CBF) becomes adept at managing high rates of concurrent insertions and deletions without the need for locking mechanisms. The integration of approximate counters within counting Bloom filters marks a significant leap in the creation of efficient concurrent data structures. By diminishing the reliance on locks and optimizing space utilization, these structures prove particularly well-suited for applications in networking and routing hardware, where concurrent access and space efficiency are pivotal considerations. However, this comes at the expense of accuracy, necessitating a delicate balance between counter precision and performance gains.

Probability of False Positives:

Consider a balls into bins approach:

m -> number of bins/relaxation

n -> number of insertions

$P_i(n,L)$ -> probability that i^{th} bin has L balls after n balls are inserted

A(n,L) -> Number of bins with greater than or equal to L balls after n balls are inserted

B(n,L) -> Number of bins with strictly less than L balls after n balls are inserted

$$B(n, L) = m - A(n, L)$$

Probability that the ith bin has Exactly L balls after n inserts is sum of the following cases:

Case 1: The ith bin has exactly L-1 balls after n-1 insertions, we select the ith bin with probability 1/m and we select a bin with greater than or equal to L-1 balls after inserting n-1 balls. In this case we will increment the ith bin and it will have exactly L balls after inserting n balls.

Case 2: The ith bin has exactly L balls after n-1 insertions, we select the ith bin with probability 1/m and we select a bin with Less than L balls after inserting n-1 balls. In this case we will not increment the ith bin and it will have exactly L balls after inserting n balls.

Case 3: The ith bin has exactly L balls after n-1 insertions, and we select any 2 bins except this one. In this case we will not increment the ith bin and it will have exactly L balls after inserting n balls.

Summing the probabilities from both the cases, we get:

$$P_i(n, L) = \frac{1}{m^2} [P_i(n - 1, L - 1). A(n - 1, L - 1) + P_i(n - 1, L). (B(n - 1, L) + (m - 1)^2)]$$

Replacing B(n,L) with n - A(n,L),

$$P_i(n, L) = \frac{1}{m^2} [P_i(n - 1, L - 1). A(n - 1, L - 1) + P_i(n - 1, L). (m - A(n, L) + (m - 1)^2)]$$

$$P_i(n, L) = \frac{1}{m^2} [P_i(n - 1, L - 1). A(n - 1, L - 1) + P_i(n - 1, L). (m^2 - m + 1 - A(n, L))]$$

From Mitzenmacher, we know,

$$\gamma_{n,L} \leq A(n, L) \leq \beta_{n,L}$$

Where,

$$\gamma_{n,i} = \frac{n}{2^{\sum_{k=0}^{i-1}(i+2-k)d^k}} \quad \beta_{n,i+4} = \frac{n}{2^{2d^i - \sum_{j=0}^{i-1} d^j}}$$

d = 2 (for two choice process)

Getting lower bound on $P_i(n,L)$:

$$P_i(n,L) \geq \frac{1}{m^2}\left[P_i(n-1, L-1).\ \gamma_{n,L-1} + P_i(n-1,L).(m^2 - m + 1 - \beta_{n,L}) \right]$$

We can generalize this for all the bins and this is becomes the probability that the relaxed counter returns Lm after n insertions

A counting bloom filter will give a false positive when each of the k counters read a value greater than the threshold value, θ :

$$P_{fp}(\theta, k, n, m) = \left(1 - \sum_{Lm<\theta} P(n, Lm)\right)^k$$

$$P_{fp}(\theta, k, n, m) \leq \left(1 - \frac{1}{m^2}\left[P_i(n-1, L-1).\ \gamma_{n,L-1} + P_i(n-1,L).(m^2 - m + 1 - \beta_{n,L}) \right]\right)^k$$

Throughput and Scalability Testing:

Similar to accuracy testing, the testing of the performance of the filer(i.e., throughput) was performed on our implementation of the relaxed concurrent counting bloom filter using 10^8 input elements. To identify how the filter's performance is impacted by various factors, we varied different factors, plotted graphs for them, and compared the results. The parameters used for testing were the size of the filter, the number of relaxed counters, and the number of threads since all of these directly affect the functioning of the filter.

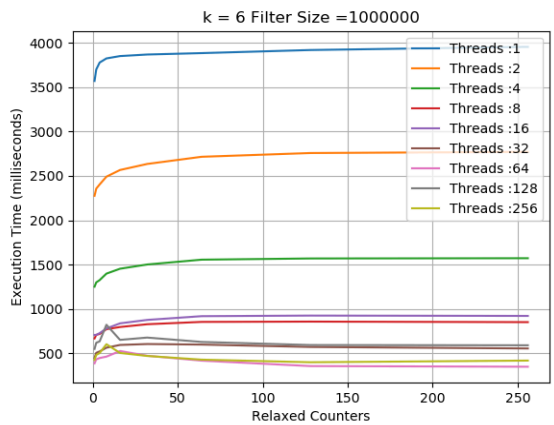
To test the raw throughput, we ignored the accuracy testing assumptions and timed the execution of each thread. Then, we took the average time out of all the threads to be our final time.

We have excluded the overhead of initializing the filter as it will be more for a larger filter size and a higher number of threads. Moreover, the initialization is not a recurrent task and only occurs once at the start. Thus it will only affect the results by a constant time if added.

Testing

The machine we used for testing had the following specifications:
CPU: AMD EPYC 7763 (2 64-core sockets with two threads per core)
Number of logical cores = 256
RAM = 512 GB

Time Testing:



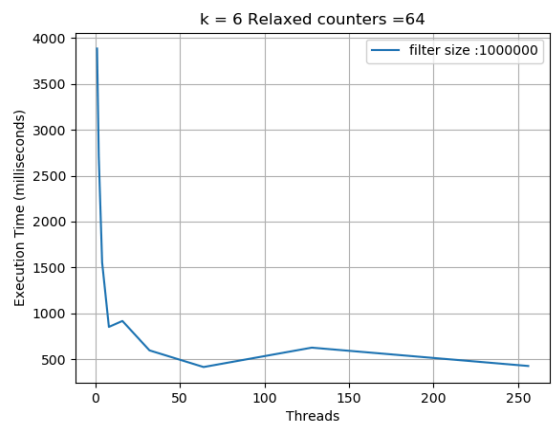
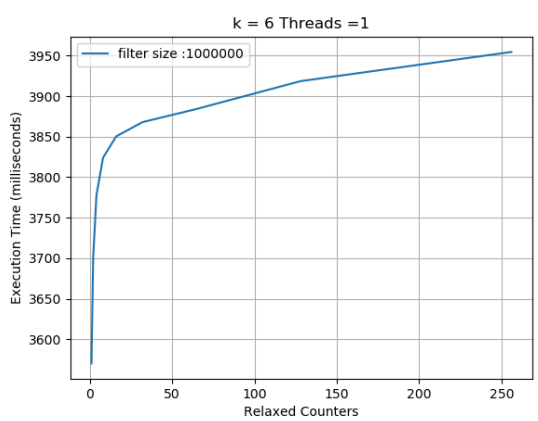


Fig: Increasing the number of threads clearly shows significant time benefit. It is clear that after a certain number of threads, the performance gain is minimal. This is due to high contention between the threads for the same memory locations.



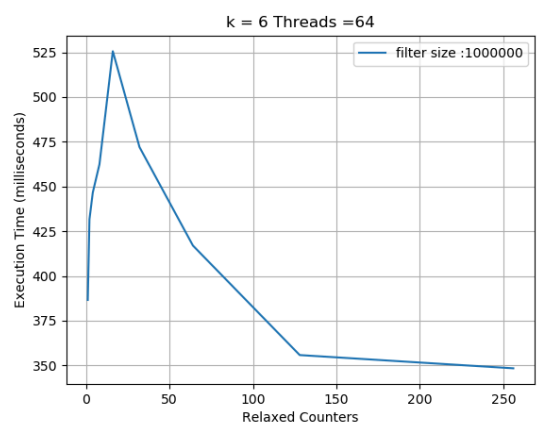


Fig: As expected, for a small number of threads, where contention is minimum, relaxing the counter produces results that are within a margin of error of each other. However, we observe the benefits of relaxation when we relax the counters to the point that many elements are being inserted into the filter simultaneously, hence causing significant contention. Due to this, there is a significant overlap between threads. As a result, relaxing the filter helps us in such cases. This suggests that if we could extend the graph beyond this, then we would observe a significant speedup.

Accuracy Measures:
In the realm of CBFs, the probability of false positives holds paramount significance, especially when approximate counters are introduced into the equation. The probability of a counter within a CBF counting 'l' elements adheres to a binomial distribution. The false positive likelihood within a CBF hinges on whether the corresponding counters for an element surpass or equal a prescribed threshold denoted as θ. Nonetheless, the incorporation of approximate counters has the potential to influence these probabilities, given that these counters may not consistently mirror the exact element count due to their relaxed update mechanism.

Accuracy Testing:
Accuracy Testing was performed on our implementation of the relaxed concurrent counting bloom filter using 10^8 elements and varied the counter size and a number of relaxed counters to gauge how accuracy changes with these values. Since accuracy is independent of the number of threads used, we chose not to use it as a parameter for our accuracy testing.

To test the filter's accuracy, we first initialized the filter by inserting some elements into it to obtain reasonable accuracy results. We randomly inserted and checked the membership of elements. We used atomic relaxed counters with the optimal number of hash functions(4; gave us the best test results). We chose to go with atomic as it showed the greatest performance in our independent testing.
Before looking at the accuracy charts, we must first understand the meaning of each term:

True Positive: A positive in a bloom filter means that the bloom filter has positively identified the object as a duplicate. Thus, a true positive is when the object is actually present in the dataset, and the bloom filter also indicates the object as duplicate.	**True Negative:** A negative in a bloom filter means that the bloom filter has identified the object as unique. A true negative is when the object is unique in the dataset, and the bloom filter also indicates the object as unique.
False Positive: A positive in a bloom filter means that the bloom filter has positively identified the object as a duplicate. Thus, a false positive is when the object is NOT actually present in the dataset, i.e., is unique, but the bloom filter still indicates the object as duplicate.	**False Negative:** A negative in a bloom filter means that the bloom filter has identified the object as unique, i.e., it did not find any existing entry. Thus, a false negative is when the object is actually present in the dataset, but the bloom filter indicates the object as unique.

Testing with Distributions:

To see how the filter performs in various scenarios, we tested it against different distributions. For this, we used two types of distributions: the Zipfian Distribution to simulate cases of peak load with a small variety of elements and the Uniform distribution to simulate cases of less load and a variety of elements. We also tried combinations of these distributions between the saturation and the testing phase. The results have been summarized below.

Zifian data inserted with Zipfian Testing:
Here, the filter was first saturated with q/2 (where q is the total number of queries) insertion queries of elements drawn from a Zipfian distribution with high skewness. Then, the remaining q/2 queries had either an insert or a delete operation called on any element belonging to the same Zipfian distribution. The results are documented below.

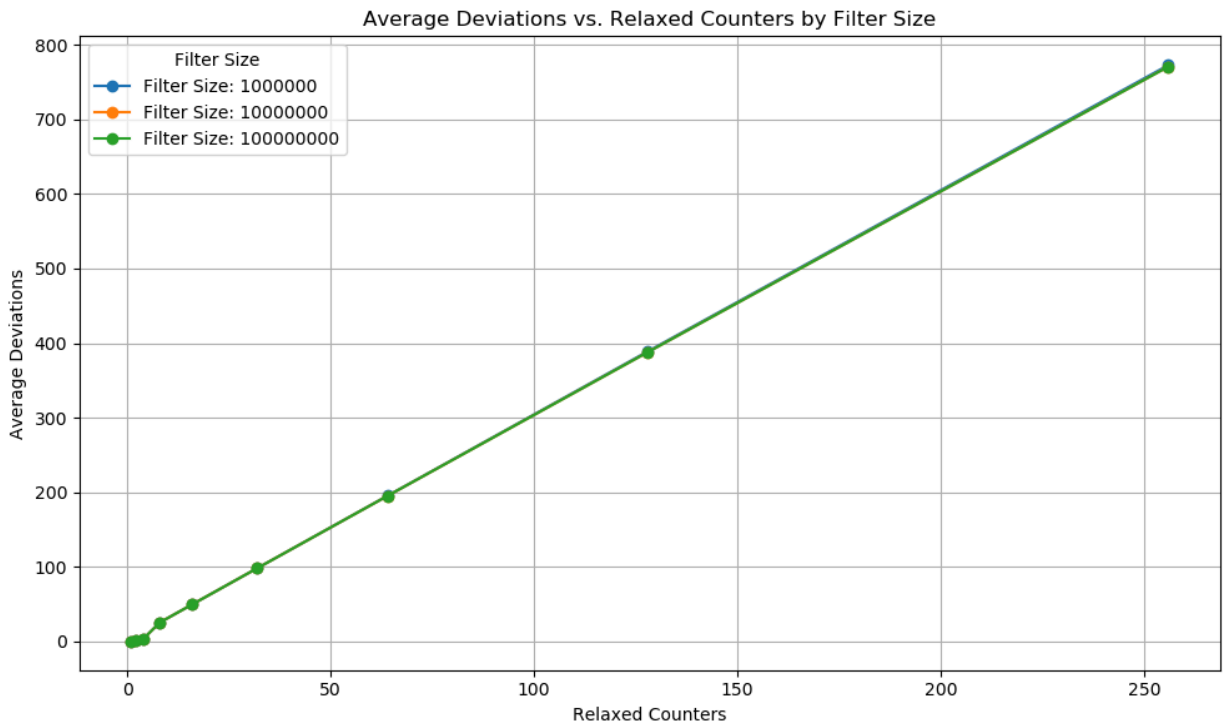
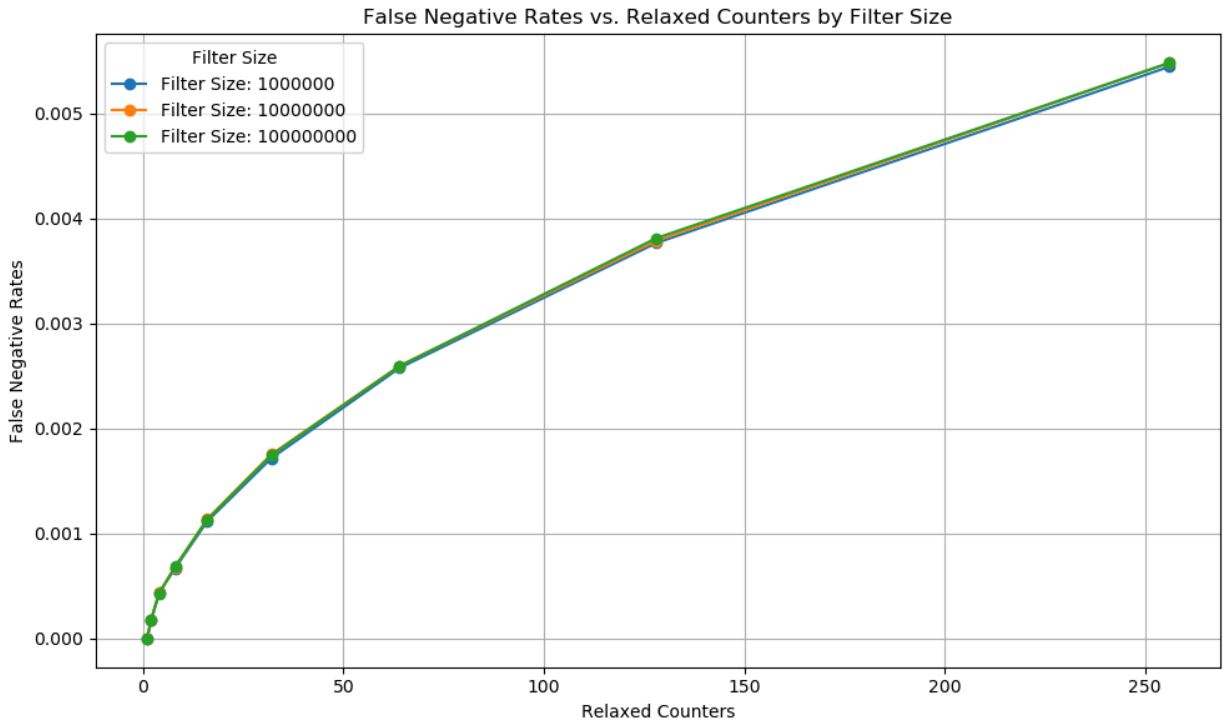


Fig: The x-axis denotes the degree of relaxation, and the y-axis represents the average deviation in frequency of the values fetched from the filter. The deviation is calculated as the difference between the actual frequency of the query element and the filter's output frequency. The average deviation is the average of deviations across all queries.
In the case of Zipfian Data with Zipfian testing, the relative error between the counters is relatively small. As a result, the deviation is a small constant multiple of the degree of relaxation. It is to be noted that the values here are a very small percentage of the element frequency.



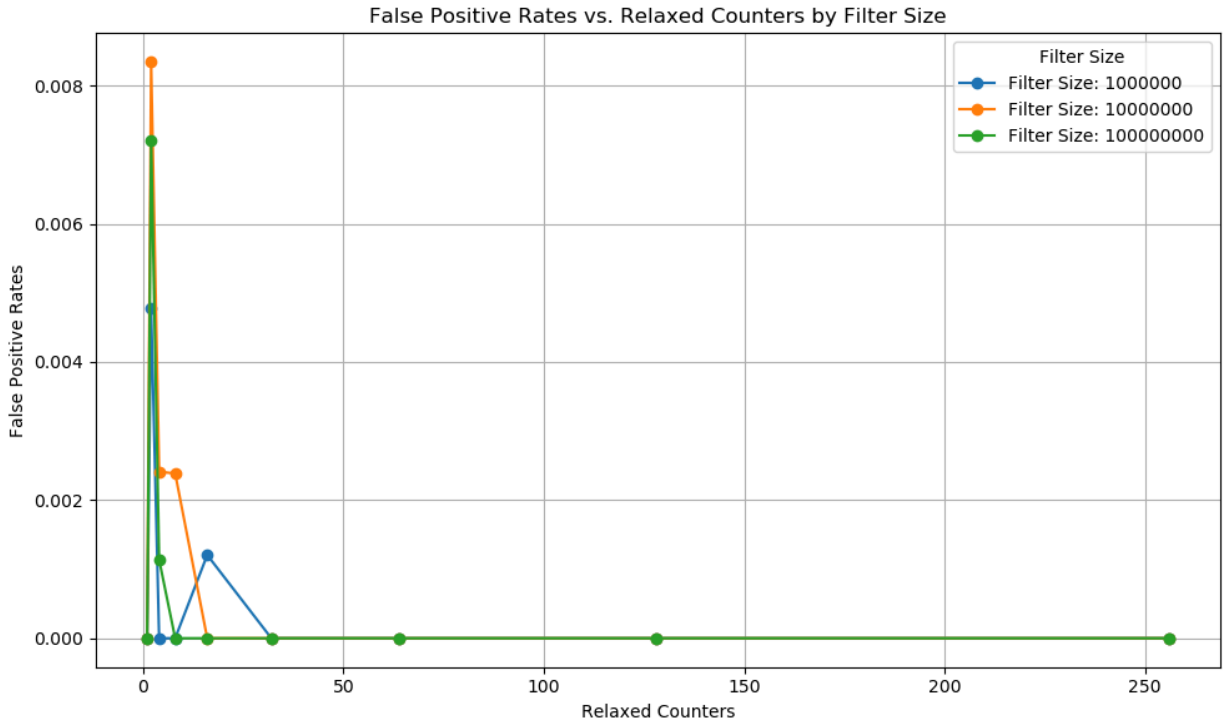


Fig: The x-axis denotes the degree of relaxation, and the y-axis represents the False Negative/Positive Rates. In the case of Zipfian Data with Zipfian testing, most values being checked in the filter are present in large numbers, which the filter can correctly predict. There are hardly any cases of true negatives, thus leading to a very low false positive and negative rate.

Zifian data with uniform testing:

Here, the filter was first saturated with q/2 (where q is the total number of queries) insertion queries of elements drawn from a Zipfian distribution with high skewness. Then, the remaining q/2 queries had either an insert or a delete operation called on any element belonging to the same Uniform distribution. The results are documented below.

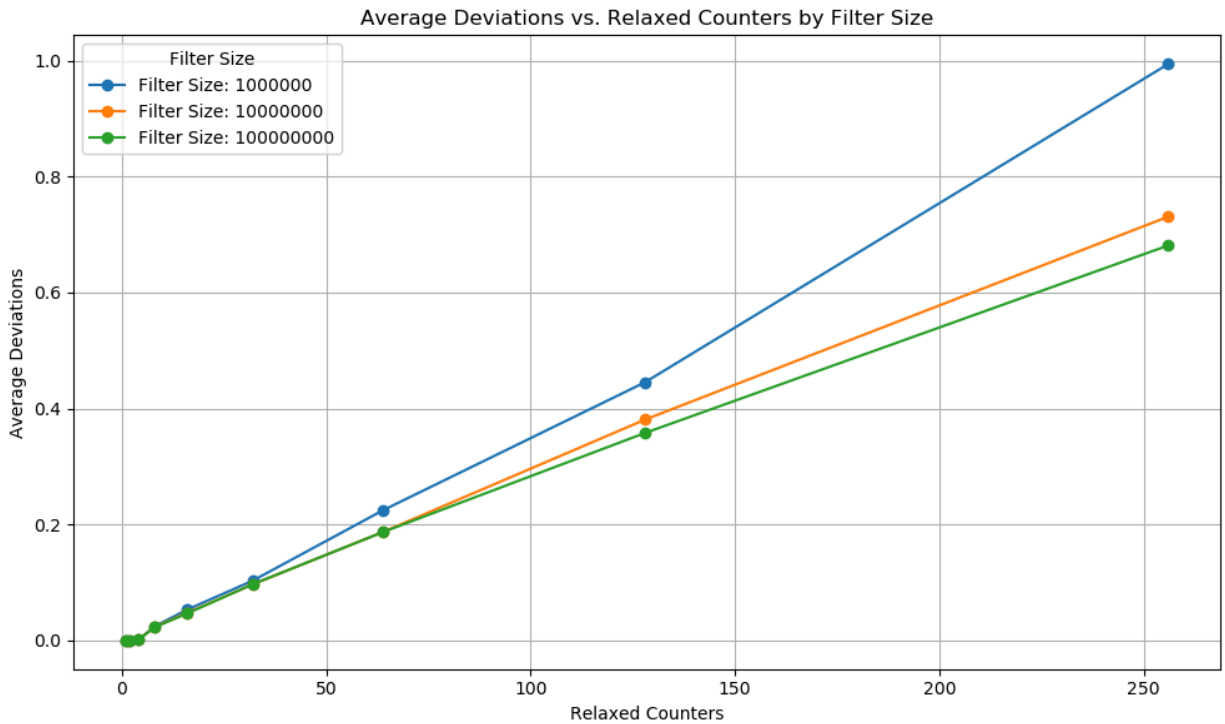
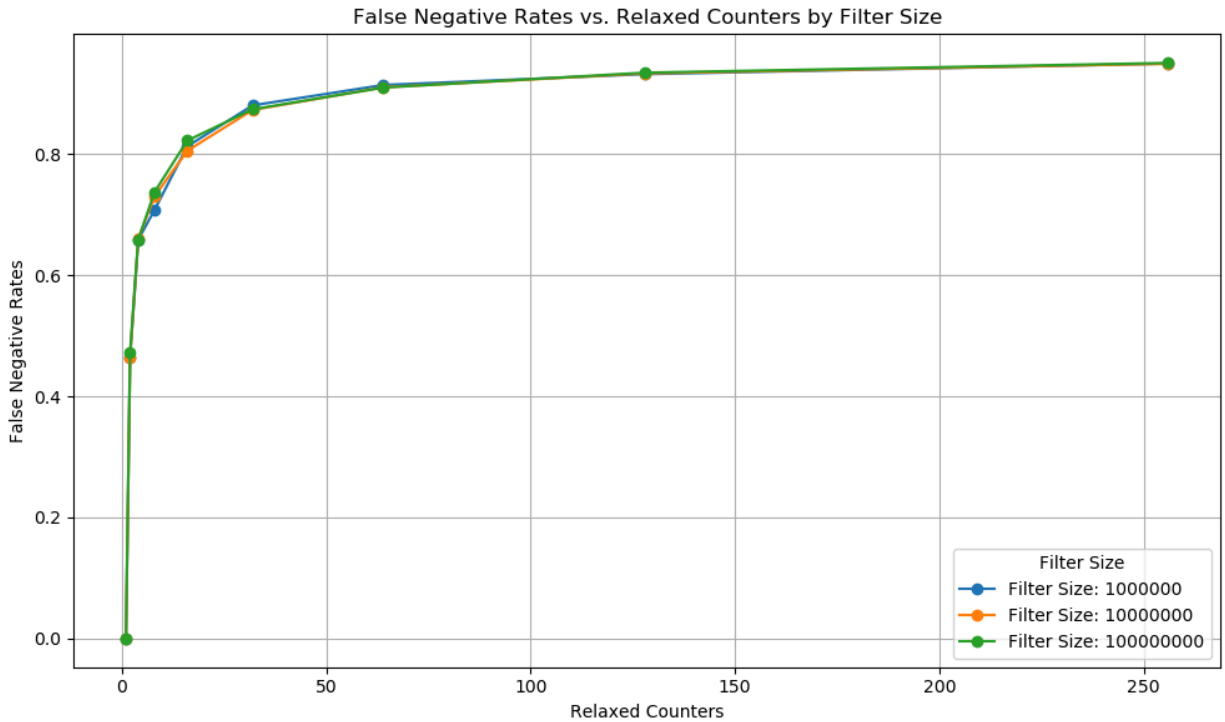


Fig: The x-axis denotes the degree of relaxation, and the y-axis represents the average deviation in frequency of the values fetched from the filter. The deviation is calculated as the difference between the actual frequency of the query element and the filter's output frequency. The average deviation is the average of deviations across all queries.

In the case of Zipfian Data with Uniform testing, most values being checked in the filter are zeros, which are returned correctly. As a result, the average is low. However, for some values, the deviation is very high but is averaged out by the 0 deviations of other values, leading to a seemingly low average deviation.



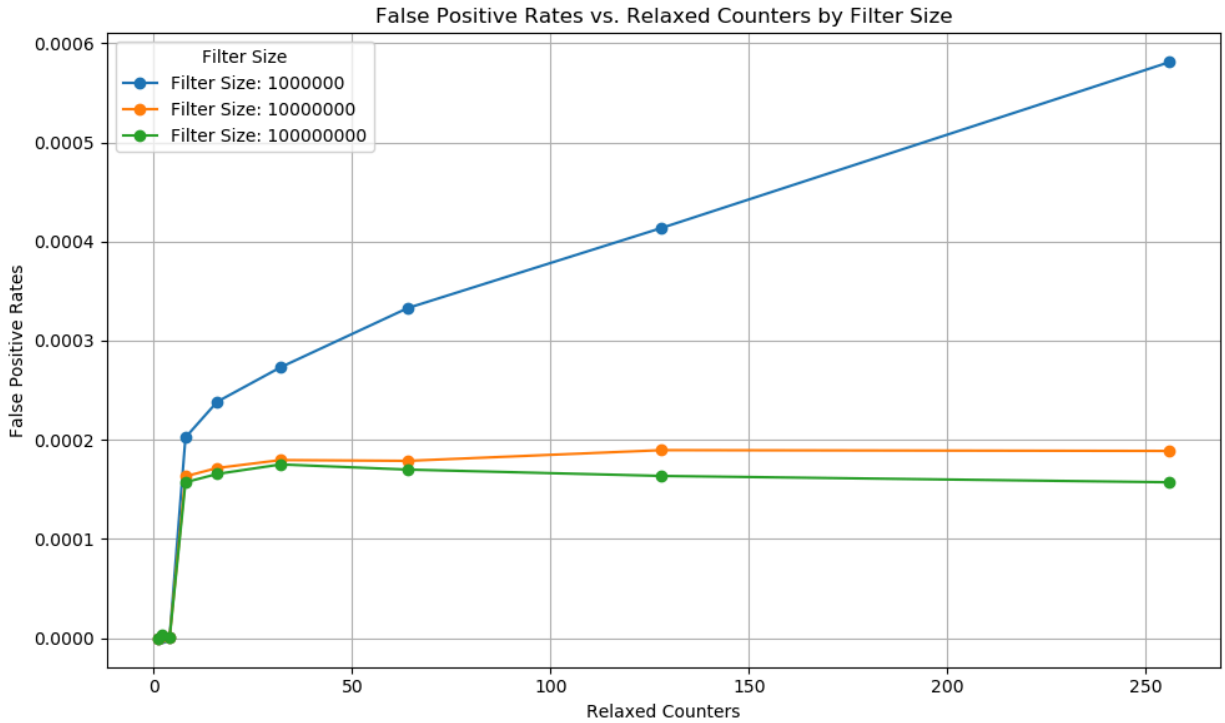


Fig: The x-axis denotes the degree of relaxation, and the y-axis represents the False Negative/Positive Rates. In the case of Zipfian Data with Uniform testing, most values being checked in the filter are absent since the filter is saturated with a Zipfian distribution, which has a small range of unique values, as a result the number of true negative values is very large, thus leading to a very low false positive and negative rate.

Uniform data with Uniform testing:

Here, the filter was first saturated with q/2 (where q is the total number of queries) insertion queries of elements drawn from a Uniform distribution with high skewness. Then, the remaining q/2 queries had either an insert or a delete operation called on any element belonging to the same Uniform distribution. The results are documented below.

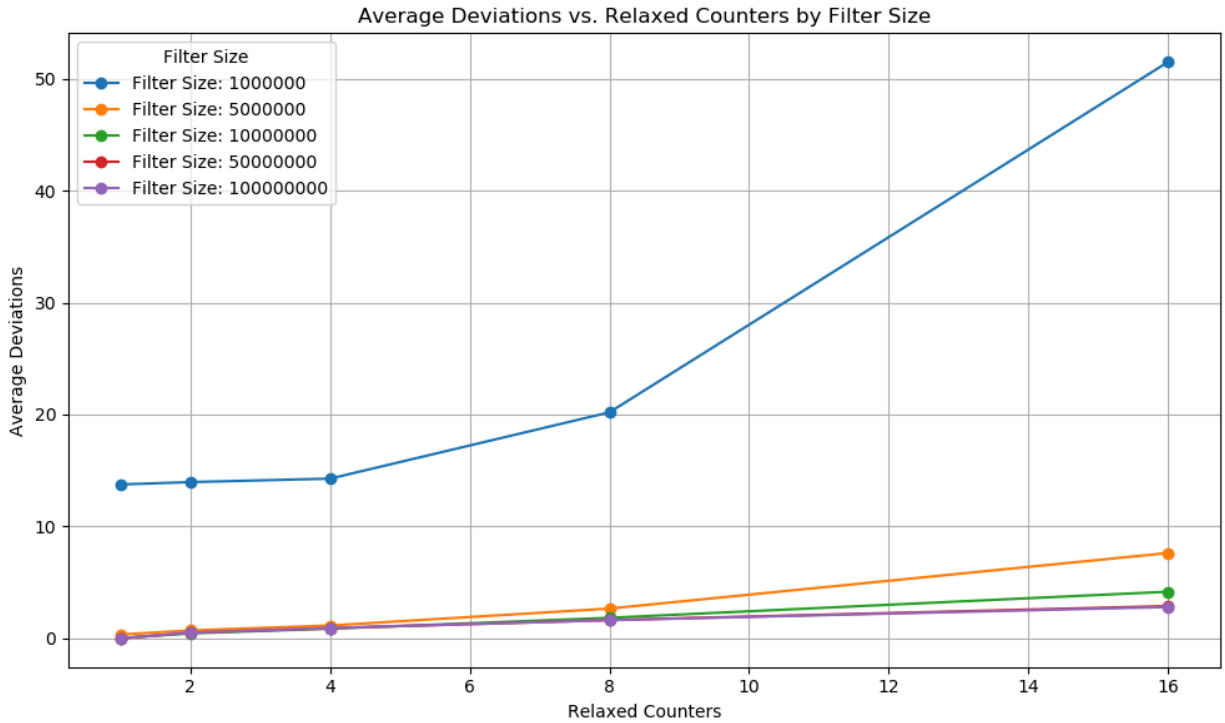
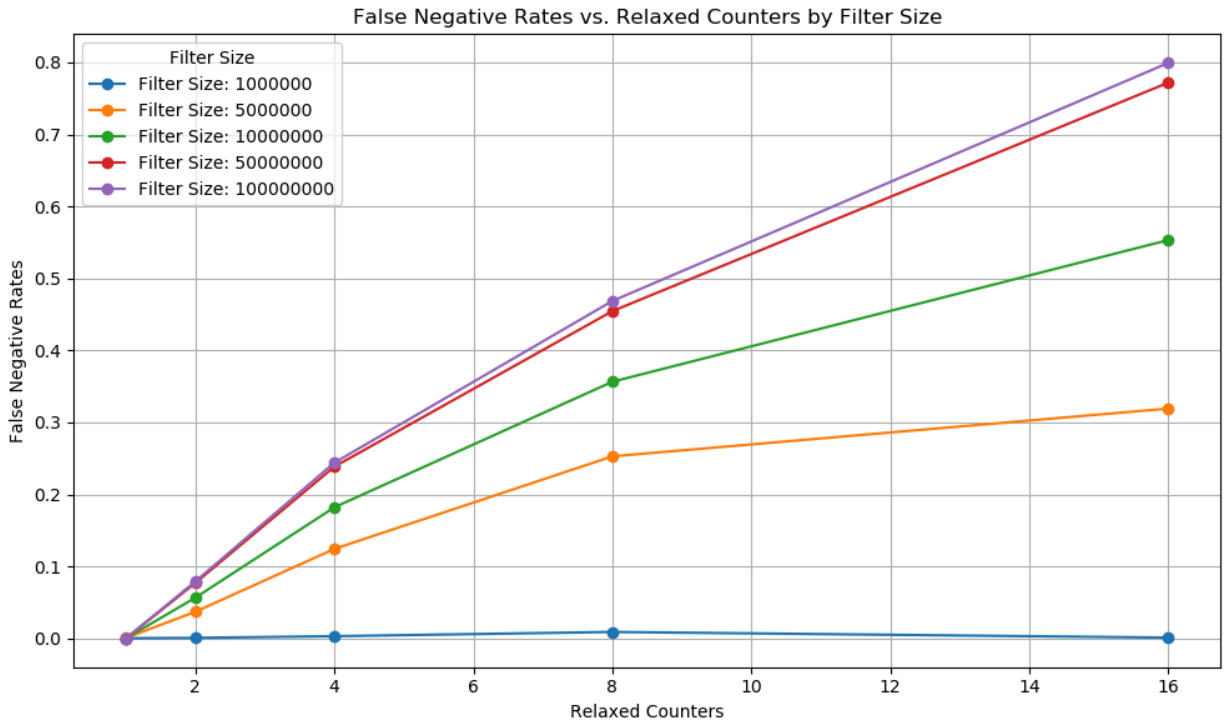


Fig: The x-axis denotes the degree of relaxation, and the y-axis represents the average deviation in frequency of the values fetched from the filter. The deviation is calculated as the difference between the actual frequency of the query element and the filter's output frequency. The average deviation is the average of deviations across all queries.

In the case of Uniform Data with Uniform testing, most values being checked in the filter are zeros, which are returned correctly. As a result, the average is low. However, for some values, the deviation is very high but is averaged out by the 0 deviations of other values, leading to a seemingly low average deviation. In the case of filters with small filter sizes, saturation occurs quickly, thus leading to the filter being saturated and giving non-zero values for most queries, thus leading to a high deviation.



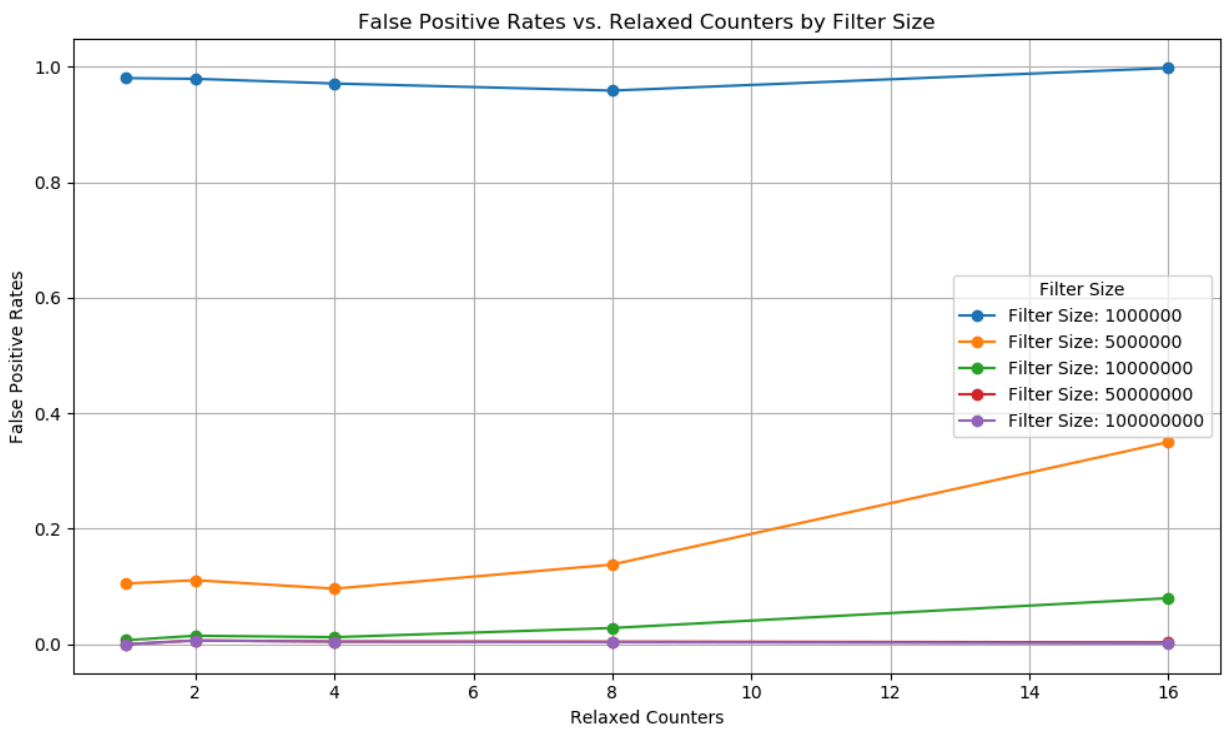


Fig: The x-axis denotes the degree of relaxation, and the y-axis represents the False Negative/Positive Rates. In the case of Uniform Data with Uniform testing, the filter has many unique elements. Still, the frequency of each element is less, which leads to poor performance of relaxed counters since they rely on high frequency to work correctly.

Future Work

In the upcoming semester and beyond, we wish to achieve the following goals:
1. **Make the bloom filter scalable:**
Since the data structure we are developing is suited for a stream setting, we wish to make it scalable. This is to prevent the accuracy bounds from changing if the size of the filter increases. A conventional counting bloom filter and even the relaxed counting bloom filter can easily saturate when a large number of elements are present and thus start giving suboptimal results. The ways that we will be investigating to prevent saturation in a bloom filter are:

- Implementing a Stable Bloom Filter: Gives more priority to recent elements
- Implementing a Scalable Bloom Filter Expands in size as the dataset expands

2. **Make it learned (deploy a small logistic regression model):** Applying a layer of classifiers can help us limit the false positive and false negative rates and reduce the time required to make a decision.

3. **Implement partitioning**: A partitioned bloom filter is another way to increase the accuracy within a bloom filter. This approach will be more useful in our case as it increases the load on some elements, thus allowing relaxation to be more evident.

Conclusion

Counting Bloom filters are highly space-efficient, with a small probability of false positives. Concurrent access to a Counting bloom filter can greatly increase its insertion and query throughput. Concurrent access results in contention between threads when they have to operate on the same counters, which can reduce the throughput. Relaxing the counters of the bloom filter can greatly reduce the contention and increase the throughput of the counting bloom filter while introducing a small probability of false negatives. The Relaxed Concurrent Counting Bloom filter seems promising for applications requiring high concurrency and greater throughput. Future work should focus on making this data structure scalable and refining its performance by including learning models.

References

Mitzenmacher, M & Upfal, Eli. (2005). Probability and Computing.

Burton H. Bloom. 1970. Space/time trade-offs in hash coding with allowable errors. Commun. ACM 13, 7 (July 1970), 422–426. https://doi.org/10.1145/362686.362692

Dan Alistarh, Trevor Brown, Justin Kopinsky, Jerry Z. Li, and Giorgi Nadiradze. 2018. Distributionally Linearizable Data Structures. In Proceedings of the 30th on Symposium on Parallelism in Algorithms and Architectures (SPAA '18). Association for Computing Machinery, New York, NY, USA, 133–142. https://doi.org/10.1145/3210377.3210411

Fan Deng and Davood Rafiei. 2006. Approximately detecting duplicates for streaming data using stable bloom filters. In Proceedings of the 2006 ACM SIGMOD international conference on Management of data (SIGMOD '06). Association for Computing Machinery, New York, NY, USA, 25–36. https://doi.org/10.1145/1142473.1142477

Flavio Bonomi, Michael Mitzenmacher, Rina Panigrahy, Sushil Singh, and George Varghese. 2006. An improved construction for counting bloom filters. In Proceedings of the 14th conference on Annual European Symposium - Volume 14 (ESA'06). Springer-Verlag, Berlin, Heidelberg, 684–695. https://doi.org/10.1007/11841036_61

Li Fan, Pei Cao, J. Almeida and A. Z. Broder, "Summary cache: a scalable wide-area Web cache sharing protocol," in IEEE/ACM Transactions on Networking, vol. 8, no. 3, pp. 281-293, June 2000, doi: 10.1109/90.851975.